대학 (567590-1) Generative Artificial Intelligence

Maximum Likelihood Learning

Spring 2026

Geonseok Lee

Department of Artificial Intelligence

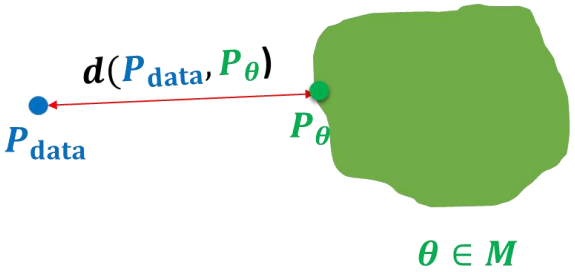
Dankook University

단국대학교 DANKOOK UNIVERSITY

Learning a generative model

- We are given a training set of examples, e.g., images of dogs

$\mathbf{x}_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$



Model family

- We want to learn a probability distribution $p(x)$ over images x such that
  - **Generation**: If we sample $x_{new} \sim p(x)$, x_{new} should look like a dog (sampling)
  - **Density estimation**: $p(x)$ should be high if x looks like a dog, and low otherwise (anomaly detection)
  - **Unsupervised representation learning**: We should be able to learn what these images have in common, e.g., ears, tail, etc. (features)
- First question: how to represent $p(x)$. Second question: **how to learn it.**

Setting

- Lets assume that the domain is governed by some underlying distribution P_{data}
- We are given a dataset $\mathcal{D}$ of m samples from P_{data}
  - Each sample is an assignment of values to (a subset of) the variables, e.g., $(X_{bank} = 1, X_{dollar} = 0, \dots, Y = 1)$ or pixel intensities.
- The standard assumption is that the data instances are **independent and identically distributed (IID)**
- We are also given a family of models $\mathcal{M}$, and our task is to learn some "good" distribution in this set:
  - For example, $\mathcal{M}$ could be all Bayes nets with a given graph structure, for all possible choices of the CPD tables
  - For example, a FVSBN (Fully Visible Sigmoid Belief Network) for all possible choices of the logistic regression parameters , θ = concatenation of all logistic regression coefficients

Goal of learning

- The goal of learning is to return a model P_θ that precisely captures the distribution P_{data} from which our data was sampled
- This is in general not achievable because of
  - limited data only provides a rough approximation of the true underlying distribution
  - computational reasons
- Example. Suppose we represent each image with a vector X of 784 binary variables (black vs. white pixel). How many possible states (= possible images) in the model? $2^{784} \approx 10^{236}$. Even 10^7 training examples provide *extremely* sparse coverage!
- We want to select P_θ to construct the ”best” approximation to the underlying distribution P_{data}
- What is ”best”?

What is “best”?

- This depends on what we want to do

① Density estimation: we are interested in the full distribution (so later we can compute whatever conditional probabilities we want)

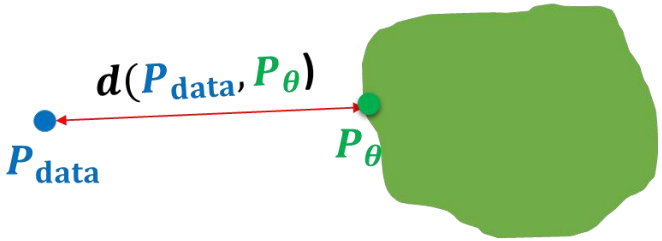
② Specific prediction tasks: we are using the distribution to make a prediction
  - Is this email spam or not?
  - **Structured prediction**: Predict next frame in a video, or caption given an image

③ Knowledge discovery: we are interested in the model itself
  - How do some genes interact with each other?
  - What causes cancer?

Learning as density estimation

- We want to learn the full distribution so that later we can answer any probabilistic inference query
- In this setting we can view the learning problem as **density estimation**
- We want to construct P_θ as ”close” as possible to P_{data} (recall we assume we are given a dataset $\mathcal{D}$ of samples from P_{data})

$\mathbf{x}_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$



$\theta \in M$

Model family

- How do we evaluate ”closeness”?

KL-divergence

- How should we measure distance between distributions?
- The **Kullback-Leibler divergence** (KL-divergence) between two distributions p and q is defined as

$$D(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

- $D(p \parallel q) \geq 0$ for all p, q, with equality if and only if $p = q$.

  Proof (Jenson's inequality):

$$E_{x \sim p}\left[-\log \frac{q(x)}{p(x)}\right] \geq -\log\left(E_{x \sim p}\left[\frac{q(x)}{p(x)}\right]\right) = -\log\left(\sum_x p(x) \frac{q(x)}{p(x)}\right) = 0$$

- Notice that KL-divergence is **asymmetric**, i.e., $D(p \parallel q) \neq D(q \parallel p)$
- Measures the expected number of extra bits required to describe *samples* from $p(x)$ using a compression code based on q instead of p

Detour on KL-divergence

- Entropy of distribution p

$$H(p) = -\sum_x p(x) \log p(x)$$

- Cross-entropy of distributions p and q

$$H(p, q) = -\sum_x p(x) \log q(x)$$

- The **Kullback-Leibler divergence** (KL-divergence)

$$\begin{aligned} D(p \parallel q) &= \sum_x p(x) \log \frac{p(x)}{q(x)} \\ &= \sum_x p(x) \log p(x) - \sum_x p(x) \log q(x) \\ &= -H(p) + H(p, q) \end{aligned}$$

- KL-divergence: if your data comes from p, but you use a scheme optimized for q, the divergence $D_{KL}(p \parallel q)$ is the number of extra bits you'll need on average

Learning as density estimation

- We want to learn the full distribution so that later we can answer any probabilistic inference query
- In this setting we can view the learning problem as **density estimation**
- We want to construct P_θ as ”close” as possible to P_{data} (recall we assume we are given a dataset $\mathcal{D}$ of samples from P_{data})
- How do we evaluate ”closeness”?
- **KL-divergence** is one possibility:

$$D(P_{data} \parallel P_\theta) = E_{x \sim P_{data}}\left[\log\left(\frac{P_{data}(x)}{P_\theta(x)}\right)\right] = \sum_x P_{data}(x) \log \frac{P_{data}(x)}{P_\theta(x)}$$

- $D(P_{data} \parallel P_\theta) = 0$ iff the two distributions are the same.
- It measures the “compression loss” (in bits) of using P_θ instead of P_{data}.

Expected log-likelihood

- We can simplify this somewhat:

$$D(P_{data} \parallel P_\theta) = E_{x\sim P_{data}}\left[\log\left(\frac{P_{data}(x)}{P_\theta(x)}\right)\right]$$
$$= E_{x\sim P_{data}}[\log P_{data}(x)] - E_{x\sim P_{data}}[\log P_\theta(x)]$$

- The first term does not depend on P_θ.
- Then, minimizing KL divergence is equivalent to maximizing the **expected log-likelihood**

$$\arg\min_{P_\theta} D(P_{data} \parallel P_\theta) = \arg\min_{P_\theta} -E_{x\sim P_{data}}[\log P_\theta(x)] = \arg\max_{P_\theta} E_{x\sim P_{data}}[\log P_\theta(x)]$$

  - Asks that P_θ assign high probability to instances sampled from P_{data}, so as to reflect the true distribution
  - Because of log, samples x where $P_\theta(x) \approx 0$ weigh heavily in objective
- Although we can now compare models, since we are ignoring $H(P_{\text{data}}) = -E_{x\sim P_{data}}[\log P_{data}(x)]$, we don't know how close we are to the optimum
- Problem: In general we do not know P_{data}

Maximum likelihood

- Approximate the expected log-likelihood

$$E_{x \sim P_{data}}[\log P_\theta(x)]$$

with the empirical log-likelihood:

$$E_{\mathcal{D}}[\log P_\theta(x)] = \frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \log P_\theta(x)$$

- **Maximum likelihood learning** is then:

$$\max_{P_\theta} \frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \log P_\theta(x)$$

- Equivalently, maximize likelihood of the data

$$P_\theta\left(x^{(1)}, \cdots, x^{(m)}\right) = \prod_{x \in \mathcal{D}} P_\theta(x)$$

Main idea in Monte Carlo Estimation

① **Express the quantity of interest as the expected value of a random variable.**

$$E_{x\sim P}[g(x)] = \sum_{x} g(x)P(x)$$

② Generate T samples $x^1, \cdots, x^T$ from the distribution P with respect to which the expectation was taken.

③ Estimate the expected value from the samples using:

$$\hat{g}(x^1, \cdots, x^T) \triangleq \frac{1}{T}\sum_{t=1}^{T} g(x^t)$$

where $x^1, \cdots, x^T$ are independent samples from P.
Note: $\hat{g}$ is a random variable. Why?

Properties of the Monte Carlo Estimate

- **Unbiased**:

$$E_P[\hat{g}] = E_P[g(x)]$$

- **Convergence**: By law of large numbers

$$\hat{g} = \frac{1}{T}\sum_{t=1}^{T} g(x^t) \rightarrow E_P[g(x)] \text{ for } T \rightarrow \infty$$

- **Variance**:

$$V_P[\hat{g}] = V_P\left[\frac{1}{T}\sum_{t=1}^{T} g(x^t)\right] = \frac{V_P[g(x)]}{T}$$

Thus, variance of the estimator can be reduced by increasing the number of samples.

Example

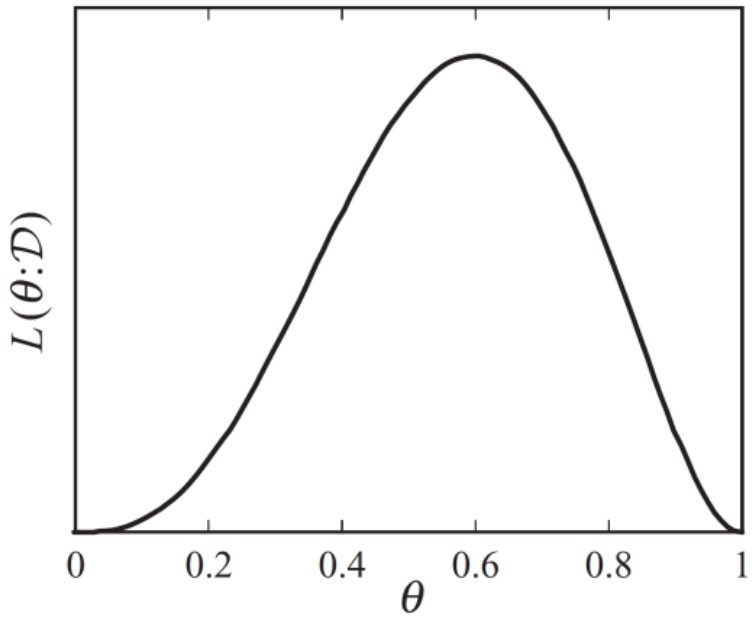
Single variable example: A biased coin

- Two outcomes: heads (H) and tails (T)
- Data set: Tosses of the biased coin, e.g., $\mathcal{D} = \{H, H, T, H, T\}$
- Assumption: the process is controlled by a probability distribution $P_{data}(x)$ where $x \in \{H, T\}$
- Class of models $\mathcal{M}$: all probability distributions over $x \in \{H, T\}$
- Example learning task: How should we choose $P_\theta(x)$ from $\mathcal{M}$ if 3 out of 5 tosses are heads in $\mathcal{D}$?

MLE scoring for the coin example

We represent our model: $P_\theta(x = H) = \theta$ and $P_\theta(x = T) = 1 - \theta$

- Example data: $\mathcal{D} = \{H, H, T, H, T\}$
- Likelihood of data $= \prod_i P_\theta(x_i) = \theta \cdot \theta \cdot (1-\theta) \cdot \theta \cdot (1-\theta)$



- Optimize for θ which makes $\mathcal{D}$ most likely. What is the solution in this case?
  $\theta = 0.6$, optimization problem can be solved in closed-form

Extending the MLE principle to autoregressive models

Given an autoregressive model with n variables and factorization

$$P_\theta(x) = \prod_{i=1}^{n} p_{\text{neural}}\left(x_i \mid x_{<i}\,;\theta_i\right)$$

$\theta = (\theta_1, \cdots, \theta_n)$ are the parameters of all the conditionals.

Training data $\mathcal{D} = \left\{x^{(1)}, \cdots, x^{(m)}\right\}$.

Maximum likelihood estimate of the parameters θ?

- Decomposition of Likelihood function

$$L(\theta, \mathcal{D}) = \prod_{i=1}^{n} P_\theta\left(x^{(j)}\right) = \prod_{j=1}^{m}\prod_{i=1}^{n} p_{\text{neural}}\left(x_i^{(j)} \mid x_{<i}^{(j)}\,;\theta_i\right)$$

- Goal: maximize $\arg\max_\theta L(\theta, \mathcal{D}) = \arg\max_\theta \log L(\theta, \mathcal{D})$

- We no longer have a closed form solution

MLE Learning: Gradient Descent

$$L(\theta, \mathcal{D}) = \prod_{i=1}^{n} P_\theta\left(x^{(j)}\right) = \prod_{j=1}^{m} \prod_{i=1}^{n} p_{\text{neural}}\left(x_i^{(j)} \mid x_{<i}^{(j)}; \theta_i\right)$$

Goal: maximize $\arg\max_\theta L(\theta, \mathcal{D}) = \arg\max_\theta \log L(\theta, \mathcal{D})$

$$\ell(\theta) = \log L(\theta, \mathcal{D}) = \sum_{j=1}^{m} \sum_{i=1}^{n} \log p_{\text{neural}}\left(x_i^{(j)} \mid x_{<i}^{(j)}; \theta_i\right)$$

① Initialize $\theta^0 = (\theta_1, \cdots, \theta_n)$ at random

② Compute $\nabla_\theta \ell(\theta)$ (by back propagation)

③ $\theta^{t+1} = \theta^t + \alpha_t \nabla_\theta \ell(\theta)$

Non-convex optimization problem, but often works well in practice

MLE Learning: Stochastic Gradient Descent

$$L(\theta, \mathcal{D}) = \prod_{i=1}^{n} P_\theta\left(x^{(j)}\right) = \prod_{j=1}^{m} \prod_{i=1}^{n} p_{\text{neural}}\left(x_i^{(j)} \mid x_{<i}^{(j)}; \theta_i\right)$$

① Initialize θ^0 at random

② Compute $\nabla_\theta \ell(\theta)$ (by back propagation)

③ $\theta^{t+1} = \theta^t + \alpha_t \nabla_\theta \ell(\theta)$

What is the gradient with respect to θ_i?

$$\nabla_\theta \ell(\theta) = \sum_{j=1}^{m} \nabla_{\theta_i} \sum_{i=1}^{n} \log p_{\text{neural}}\left(x_i^{(j)} \mid x_{<i}^{(j)}; \theta_i\right) = \sum_{j=1}^{m} \nabla_{\theta_i} \log p_{\text{neural}}\left(x_i^{(j)} \mid x_{<i}^{(j)}; \theta_i\right)$$

Each conditional $p_{\text{neural}}(x_i \mid x_{<i}; \theta_i)$ can be optimized separately if there is no parameter sharing.

In practice, parameters θ_i are shared (e.g., NADE, PixelRNN, PixelCNN, etc.)

MLE Learning: Stochastic Gradient Descent

$$L(\theta, \mathcal{D}) = \prod_{i=1}^{n} P_\theta\left(x^{(j)}\right) = \prod_{j=1}^{m} \prod_{i=1}^{n} p_{\text{neural}}\left(x_i^{(j)} \mid x_{<i}^{(j)}; \theta_i\right)$$

① Initialize θ^0 at random

② Compute $\nabla_\theta \ell(\theta)$ (by back propagation)

③ $\theta^{t+1} = \theta^t + \alpha_t \nabla_\theta \ell(\theta)$

$$\nabla_\theta \ell(\theta) = \sum_{j=1}^{m} \sum_{i=1}^{n} \nabla_\theta \log p_{\text{neural}}\left(x_i^{(j)} \mid x_{<i}^{(j)}; \theta_i\right)$$

What if $m = |\mathcal{D}|$ is huge?

$$\nabla_\theta \ell(\theta) = m \sum_{j=1}^{m} \frac{1}{m} \sum_{i=1}^{n} \nabla_\theta \log p_{\text{neural}}\left(x_i^{(j)} \mid x_{<i}^{(j)}; \theta_i\right)$$

$$= m E_{x^{(j)} \sim \mathcal{D}}\left[\sum_{i=1}^{n} \nabla_\theta \log p_{\text{neural}}\left(x_i^{(j)} \mid x_{<i}^{(j)}; \theta_i\right)\right]$$

Monte Carlo: Sample $x^{(j)} \sim \mathcal{D}$; $\nabla_\theta \ell(\theta) \approx m \sum_{i=1}^{n} \nabla_\theta \log p_{\text{neural}}\left(x_i^{(j)} \mid x_{<i}^{(j)}; \theta_i\right)$

Empirical Risk and Overfitting

- Empirical risk minimization can easily **overfit** the data
  - Extreme example: The data is the model (remember all training data).
- Generalization: the data is a sample, usually there is vast amount of samples that you have never seen. Your model should generalize well to these “never-seen” samples.
- Thus, we typically restrict the **hypothesis space** of distributions that we search over

Bias-Variance trade off

- If the hypothesis space is very limited, it might not be able to represent P_{data}, even with unlimited data
  - This type of limitation is called **bias**, as the learning is limited on how close it can approximate the target distribution
- If we select a highly expressive hypothesis class, we might represent better the data
  - When we have small amount of data, multiple models can fit well, or even better than the true model. Moreover, small perturbations on $\mathcal{D}$ will result in very different estimates
  - This limitation is call the **variance**.

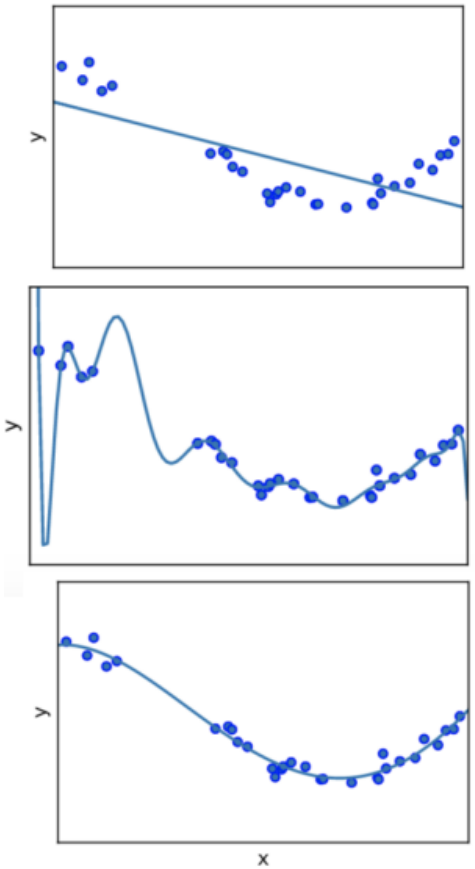
Bias-Variance trade off

- There is an inherent **bias-variance trade off** when selecting the hypothesis class. Error in learning due to both things: bias and variance.

- Hypothesis space: linear relationship
  - Does it fit well? Underfits

- Hypothesis space: high degree polynomial
  - Overfits

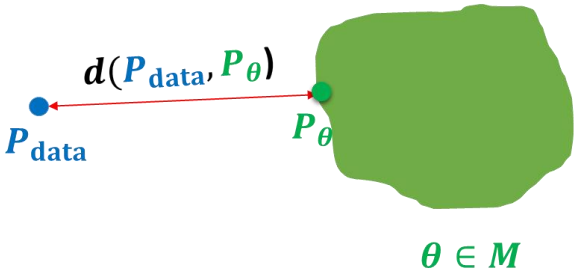
- Hypothesis space: low degree polynomial
  - Right tradeoff



How to avoid overfitting?

- Hard constraints, e.g. by selecting a less expressive model family:
  - Smaller neural networks with less parameters
  - Weight sharing

$\mathbf{x}_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$



- Soft preference for “simpler” models: Occam Razor.
- Augment the objective function with regularization:

$$\text{objective}(x, \mathcal{M}) = \text{loss}(x, \mathcal{M}) + R(\mathcal{M})$$

- Evaluate generalization performance on a held-out validation set

Recap

- For autoregressive models, it is easy to compute $p_\theta(x)$
  - Ideally, evaluate in parallel each conditional log $p_{\text{neural}}\left(x_i^{(j)} \mid x_{<i}^{(j)}\,;\theta_i\right)$
    Not like RNNs.
- Natural to train them via maximum likelihood
- Higher log-likelihood doesn't necessarily mean better looking samples
- Other ways of measuring similarity are possible (Generative Adversarial Networks, GANs)